

COMPILER DESIGN – UNIT III

Part 2: Intermediate Code Generation

Student-Friendly | Exam-Oriented Notes

1. Intermediate Code Generation

Intermediate Code Generation is the compiler phase that converts the source program into an intermediate representation (IR) before generating target/machine code.

```
Source Program
    ↓
Lexical Analysis
    ↓
Syntax Analysis
    ↓
Semantic Analysis
    ↓
Intermediate Code
    ↓
Code Optimization
    ↓
Target Code
```

Advantages:

- Makes compiler design easier.
- Generally machine independent.
- Helps in code optimization.
- The same IR can be translated to different target machines.

Example:

```
a = b + c * d

t1 = c * d
t2 = b + t1
a = t2
```

2. Variants of Syntax Trees

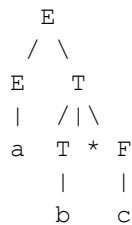
A syntax tree represents the structure of an expression or program.

Example: $a + b * c$

```
      +
     / \
    a  *
       / \
      b  c
```

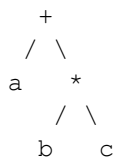
2.1 Parse Tree

A parse tree contains grammar symbols and shows the complete derivation.



2.2 Abstract Syntax Tree (AST)

An AST removes unnecessary grammar symbols and keeps the essential structure.



Parse Tree vs AST:

Parse Tree	AST
Contains grammar symbols	Removes unnecessary grammar symbols
Usually larger	Usually smaller
Shows complete derivation	Shows essential structure
Mainly useful in syntax analysis	Useful for semantic analysis and code generation

3. Three-Address Code (TAC)

Three-Address Code is an intermediate representation in which each instruction contains at most three addresses.

$x = y \text{ op } z$

Here: x = result, y = first operand, z = second operand, op = operator.

Common TAC Instructions

Instruction	Example
Binary operation	t1 = a + b
Unary operation	t1 = -a
Copy	x = y
Conditional jump	if x < y goto L1
Unconditional jump	goto L1
Parameter	param x
Procedure call	t1 = call fun, 2
Return	return x

4. TAC Example

For: $a = b * c + d$

```
t1 = b * c
t2 = t1 + d
a = t2
```

The complex expression is broken into simple operations using temporary variables.

5. Types and Declarations

A type tells the compiler what kind of value a variable can store.

```
int age;
float salary;
char grade;
```

- age → int
- salary → float
- grade → char

Types help the compiler allocate memory, perform type checking, detect errors, and generate suitable target code.

5.1 Type Expressions

A type expression represents the type of a variable, expression, or function.

```
int
float
char
```

Array example:

```
int a[10]
Type expression: array(10, int)
```

Function example:

```
int add(int a, int b)
Type expression: int × int → int
```

Important: Here × does NOT mean multiplication. It means the two input types taken together. → means 'produces' or 'returns'.

```
int × int → int
= two integer inputs → one integer result
```

For example, int add(int a, int b) takes two integers and returns an integer.

```
5 + 10 = 15
int × int → int
```

Another example:

```
float average(int a, int b)
Type expression: int × int → float
```

Exam sentence: 'int × int → int represents a function that takes two integer arguments and returns an integer value.'

6. Type Checking

Type checking is the process of checking whether operands and expressions have compatible data types. It is part of semantic analysis.

Correct:

```
int a;
a = 10;

int = int → Valid
```

Type error:

```
int a;
a = "hello";

int ≠ string → Type Error
```

Expression example:

```
int a, b, c;
c = a + b;

a → int, b → int
therefore a + b → int
c → int
Assignment is valid.
```

7. Control Flow

Control flow describes the order in which program statements are executed.

Sequential execution:

```
a = 10;
b = 20;
c = a + b;

S1 → S2 → S3
```

Conditional statement:

```
if (a < b)
    x = 10;
else
    x = 20;

if a < b goto L1
goto L2
L1:
x = 10
goto L3
L2:
x = 20
L3:
```

Loop:

```
while (a < 10)
    a = a + 1;

L1:
if a < 10 goto L2
goto L3
L2:
t1 = a + 1
a = t1
goto L1
L3:
```

8. Basic Blocks

A basic block is a sequence of consecutive statements with one entry point and one exit point, with no branching inside the block.

```
a = 10
b = 20
c = a + b
```

Example:

```
1. a = 10
2. b = 20
3. c = a + b
4. if c > 20 goto L1
5. d = 5
6. goto L2
7. L1: d = 10
8. L2: print(d)
```

Basic blocks:

```
B1: a = 10; b = 20; c = a + b; if c > 20 goto L1
B2: d = 5; goto L2
B3: L1: d = 10
B4: L2: print(d)
```

9. Backpatching

Backpatching is a technique used to fill in the target addresses or labels of jumps when their destinations are not known when the intermediate code is generated.

```
Initially:
if a < b goto ____
```

```
Later:
if a < b goto L1
```

Used especially for Boolean expressions, if-else, while/for loops, and short-circuit evaluation.

List	Meaning
truelist	Instructions that jump when the condition is true

falselist	Instructions that jump when the condition is false
nextlist	Instructions that jump to the next statement

truelist → TRUE destination
 falselist → FALSE destination
 nextlist → NEXT statement

10. Switch Statements

A switch statement allows selection among multiple alternatives.

```

switch(x)
{
  case 1: y = 10; break;
  case 2: y = 20; break;
  default: y = 0;
}

```

Possible intermediate code:

```

if x == 1 goto L1
if x == 2 goto L2
goto L3
L1:
y = 10
goto L4
L2:
y = 20
goto L4
L3:
y = 0
L4:

```

Two common implementations:

- Sequence of conditional jumps.
- Jump table – useful when there are many suitable/consecutive case values.

11. Intermediate Code for Procedures

A procedure/function is a block of code that can be called from another part of a program. IR represents parameters, calls, operations, and return values.

```

int add(int a, int b)
{
  return a + b;
}

```

Possible TAC:

```

add:
t1 = a + b
return t1

```

For $x = \text{add}(p, q)$:

```

param p
param q
t1 = call add, 2
x = t1

```

- param → passes an argument
- call → calls the procedure/function
- 2 → number of arguments
- return → returns a value

12. Quick Revision

Topic	Simple meaning
Intermediate Code	Code between source and target code
Syntax Tree	Represents program structure
AST	Simplified syntax tree
TAC	Intermediate code using simple instructions
Type	Specifies the kind of data
Type Checking	Checks compatible data types
Control Flow	Order of statement execution
Basic Block	Straight-line sequence of statements
Backpatching	Filling unknown jump targets later
Switch	Multiple-choice selection
Procedure Code	IR for calls, parameters and returns

13. Important Exam Questions

5 Marks

- Explain intermediate code generation with its advantages.
- Explain Abstract Syntax Tree with an example.
- Explain Three-Address Code and its common instructions.
- Explain type checking with examples.
- What is a basic block?
- Explain backpatching and its lists.
- Explain intermediate code for switch statements.
- Explain intermediate code for procedure calls.

10 Marks

- Explain Three-Address Code with suitable examples and different types of TAC instructions.
- Explain control flow and basic blocks with an example.
- Explain backpatching with truelist, falselist and nextlist.
- Generate intermediate code for an if-else/while statement.
- Explain intermediate code generation for switch statements and procedures.

★ **Exam Focus: TAC → Basic Blocks/Control Flow → Backpatching → Switch → Procedures → Type Checking**